

DummyJSON API — OAuth Login & JWT Bearer Token Testing Guide

Base URL: `https://dummyjson.com` **Auth Type:** Bearer JWT | **Tool:** Postman |
Version: 1.0 **Content-Type:** `application/json` **Scope:** OAuth Login · JWT
Extraction · Authenticated Endpoint Testing

Contents

- DummyJSON API — OAuth Login & JWT Bearer Token Testing Guide
 - Overview
 - Request Chain
 - Create Environment & Collection
 - POST Login API
 - Extract Access Token via Script
 - Verify Access Token
 - Set Bearer Token Authorization
 - Confirm Authorization Header
 - GET User Profile
 - GET Products
 - Save Product ID via Script
 - Verify Environment Variables
 - GET Product by ID
 - HTTP Status Code Reference
-

Overview

This guide covers OAuth login and JWT Bearer token testing for the DummyJSON API using Postman. It includes environment setup, token extraction via post-response scripts,

and authenticated requests for user and product data.

Property	Value
Base URL	<code>https://dummyjson.com</code>
Auth Type	Bearer JWT
Content-Type	<code>application/json</code>
Tool	Postman (Desktop or Web)
Environment	<code>Login_Environment</code>
Collection	<code>OAuth-Login</code>

Prerequisites

Requirement	Details
Postman installed	Desktop or Web version
Internet access	Reach <code>https://dummyjson.com</code>

Steps at a Glance

Step	Action	Type
1	Create Environment & Collection	Setup
2	POST Login	POST
3	Extract Access Token via Script	Script
4	Verify Access Token	Verify
5	Set Bearer Token Authorization	Auth
6	Confirm Authorization Header	Header
7	GET User Profile	GET
8	GET Products	GET
9	Save Product ID via Script	Script
10	Verify Environment Variables	Verify
11	GET Product by ID	GET

✓ **Best Practice:** Store `BASE_URL`, `accessToken`, and `productId` in `Login_Environment`. This removes hardcoded values and passes tokens automatically between requests.

Request Chain

The following table lists each step, its action, and its key output.

#	Action	Key Output
1	Create <code>Login_Environment</code> ; set <code>BASE_URL</code>	<code>BASE_URL</code> = <code>https://dummyjson.com</code>
2	POST <code>/auth/login</code> with credentials	<code>accessToken</code> + <code>refreshToken</code> returned
3	Post-response script saves <code>accessToken</code>	<code>accessToken</code> stored in <code>Login_Environment</code>
4	Open <code>Login_Environment</code>	Confirm <code>accessToken</code> is populated
5	Set Auth Type to Bearer Token; enter <code>{{accessToken}}</code>	Authorization configured on request
6	Check Headers tab	<code>Authorization: {{accessToken}}</code> auto-populated
7	GET <code>/auth/me</code>	Full user profile · 200 OK
8	GET <code>/auth/products</code>	Products array · 200 OK
9	Post-response script saves <code>products[0].id</code>	<code>productId</code> stored in <code>Login_Environment</code>
10	Open <code>Login_Environment</code>	Confirm all three variables are set
11	GET <code>/auth/products/{{productId}}</code>	Single product · 200 OK

Create Environment & Collection

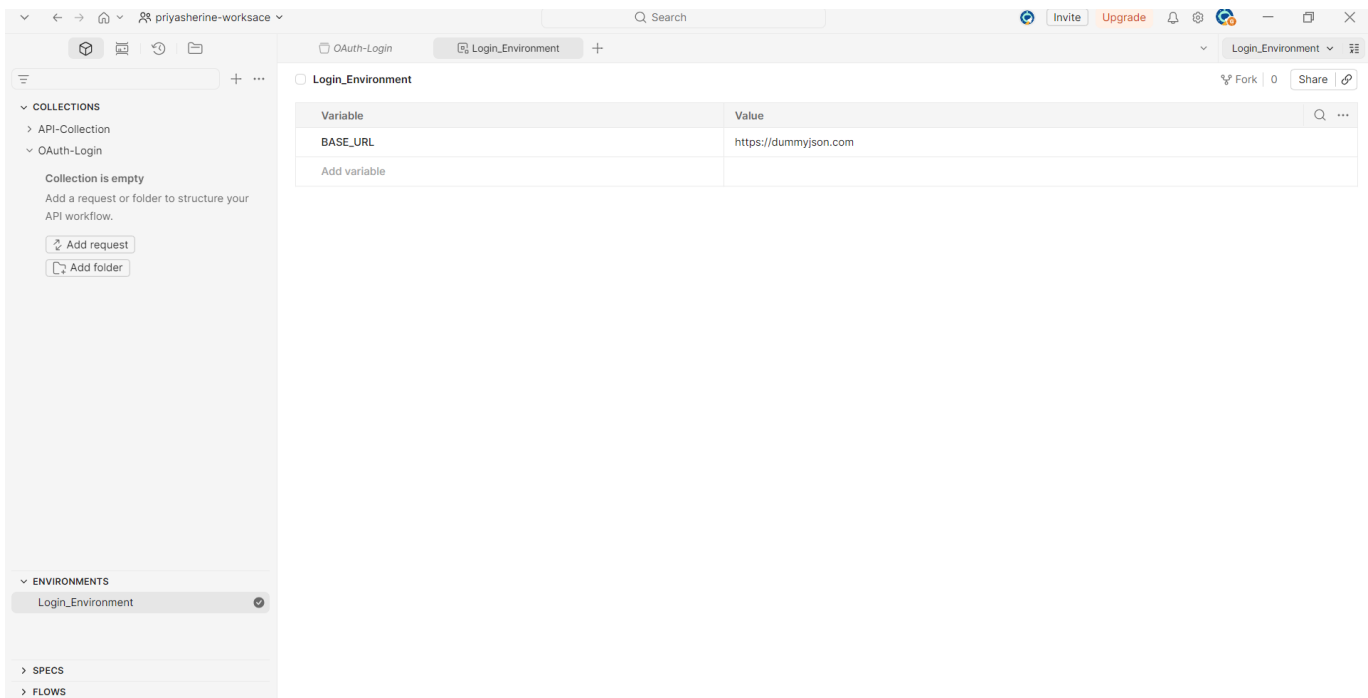
1 Setup — Create the Postman environment and collection

Procedure

1. Click **Environments** in the left sidebar.
2. Click **+**. Name the environment `Login_Environment`.
3. In the **Variable** column, enter `BASE_URL`.
4. In the **Value** column, enter `https://dummyjson.com`.
5. Click **Save**.
6. In the top-right selector, select `Login_Environment`.
7. Click **Collections** → **+**. Name the collection `OAuth-Login`.

Screenshot 1

`Login_Environment` — `BASE_URL` = `https://dummyjson.com`



POST Login API

POST `{{BASE_URL}}/auth/login` — Authenticate and receive a JWT access token

Request Details

Property	Value
Method	POST
URL	<code>{{BASE_URL}}/auth/login</code>
Body Type	raw · JSON
Auth	None
Expected	200 OK

Request URL

```
https://dummyjson.com/auth/login
```

Procedure

1. In `OAuth-Login`, click **Add request**. Name it `Login API`.
2. Set the method to **POST**.
3. Enter the URL: `{{BASE_URL}}/auth/login`
4. Click the **Body** tab. Select **raw** → **JSON**.
5. Paste the request body below.
6. Click **Send**. Confirm `200 OK`.

Request Body

```
{  
  "username": "emilys",
```

```
"password": "emilypass"
}
```

Example Response

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "id": 1,
  "username": "emilys",
  "email": "emily.johnson@x.dummyjson.com",
  "firstName": "Emily",
  "lastName": "Johnson",
  "gender": "female",
  "image": "https://dummyjson.com/icon/emilys/128"
}
```

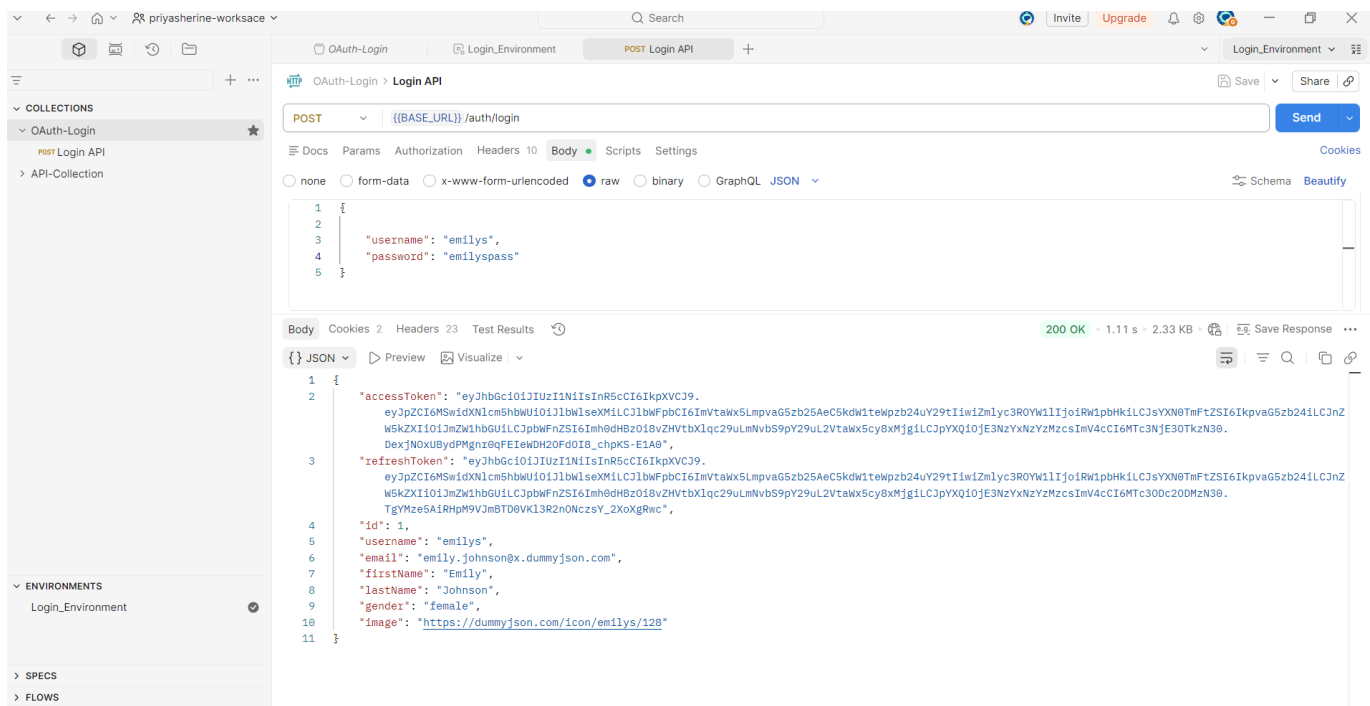
Key Response Fields

Field	Type	Description
accessToken	String (JWT)	Short-lived token for authenticating subsequent requests.
refreshToken	String (JWT)	Long-lived token for generating a new access token on expiry.
id	Int	Unique identifier of the authenticated user.
username	String	Username of the authenticated user.
email	String	Email address of the authenticated user.
firstName	String	User's first name.
lastName	String	User's last name.

Screenshot 2

POST /auth/login — 200 OK · accessToken and refreshToken in

response



Extract Access Token via Script

3

Script — Save the accessToken to the environment automatically

Procedure

1. Open the `Login API` request.
2. Click the **Scripts** tab → **Post-response**.
3. Paste the script below.
4. Click **Save**, then **Send**.
5. Open the **Postman Console**. Confirm the token prints.

Post-Response Script

```
// Parse the response body
let jsonData = pm.response.json();

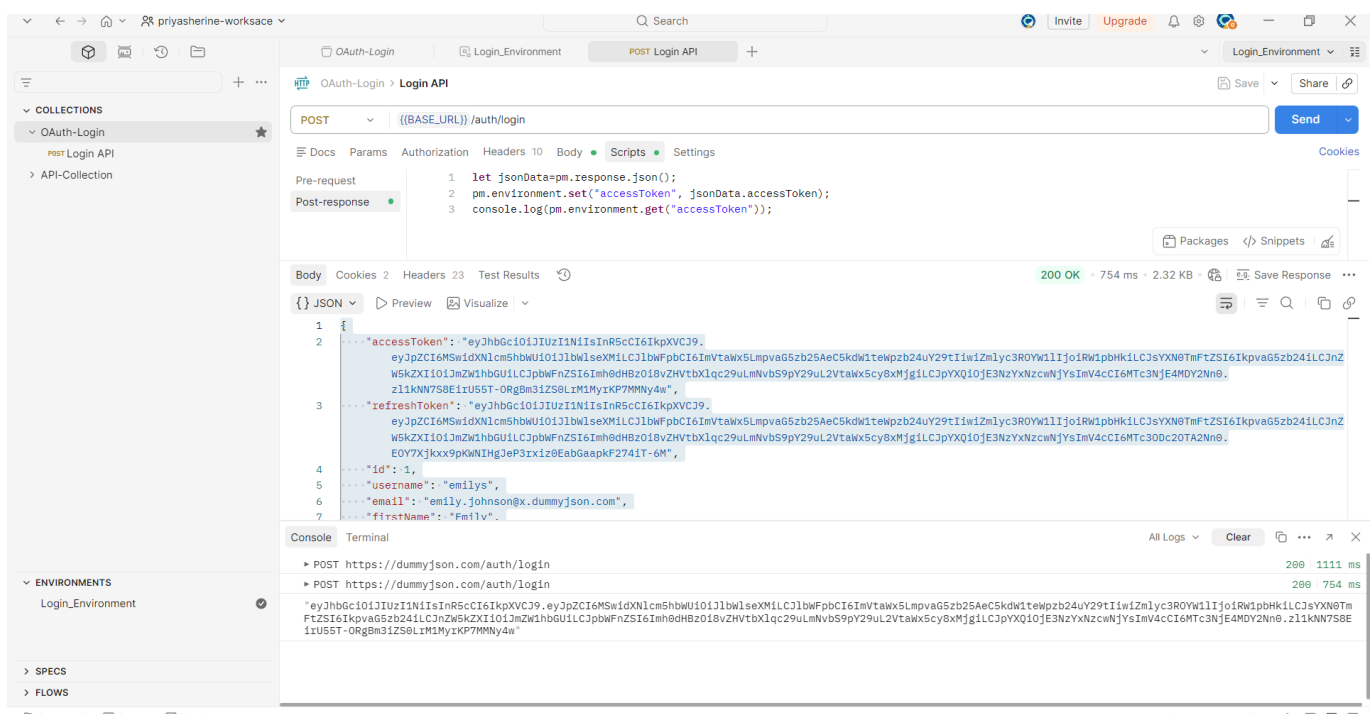
// Save accessToken to the active environment
pm.environment.set("accessToken", jsonData.accessToken);

// Log the token to the Postman Console
console.log(pm.environment.get("accessToken"));
```

💡 The Console panel confirms the token is set before you proceed to authenticated requests.

Screenshot 3

Scripts tab — Post-response script saves `accessToken` to `Login_Environment`



Verify Access Token

4

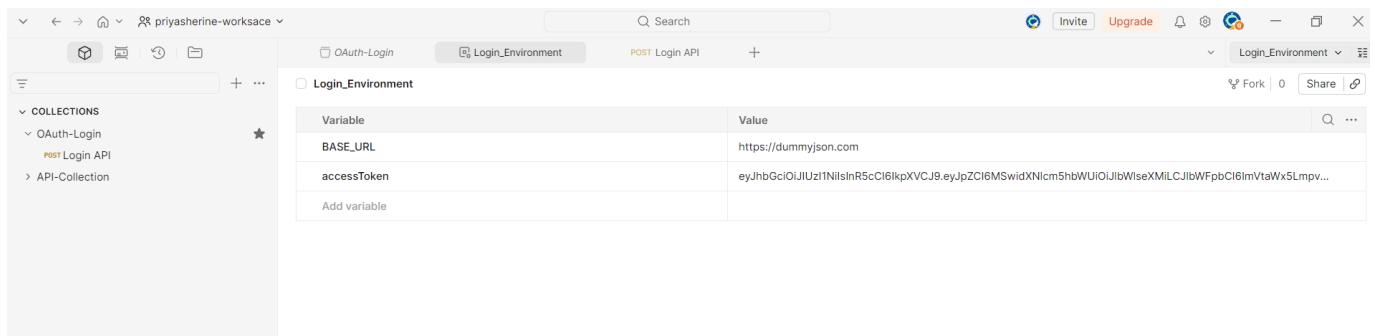
Verify — Confirm the `accessToken` is stored in the environment**Procedure**

1. Click **Environments** in the left sidebar.
2. Click **Login_Environment**.
3. Confirm the `accessToken` row shows a JWT value in the **Value** column.

⚠ If `accessToken` is empty, the script did not run. Return to **Extract Access Token via Script**, confirm the script is in the **Post-response** tab, and click **Send** again.

Screenshot 4

`Login_Environment` — `accessToken` populated with JWT value



Set Bearer Token Authorization

5

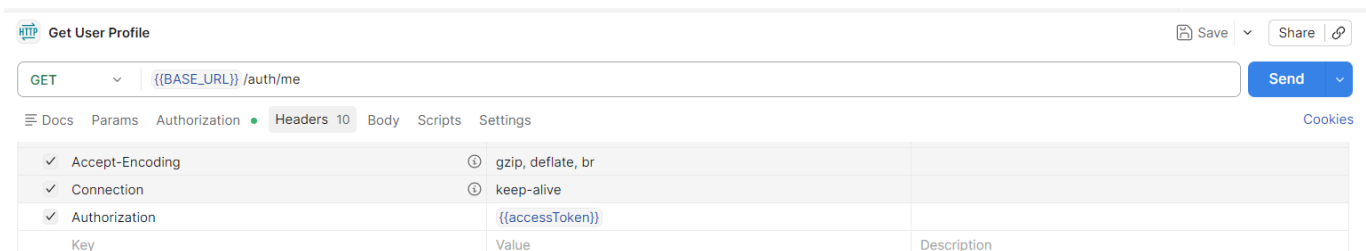
Auth — Configure the Bearer Token on the Get User Profile request**Procedure**

1. In `OAuth-Login`, click **Add request**. Name it `Get User Profile`.
2. Set the method to **GET**. Enter the URL: `{{BASE_URL}}/auth/me`
3. Click the **Authorization** tab.
4. In **Auth Type**, select **Bearer Token**.
5. In **Token**, enter `{{accessToken}}`.
6. Click **Save**.

i Postman generates the `Authorization: Bearer <token>` header automatically. No manual header entry is needed.

Screenshot 5

Authorization tab — Auth Type: Bearer Token · Token: `{{accessToken}}`



Confirm Authorization Header

6 Header — *Confirm the Authorization header is auto-populated*

Procedure

1. On `Get User Profile`, click the **Headers** tab.
2. Confirm `Authorization` shows `{{accessToken}}` in the **Value** column.

Expected Headers

Header	Value
Authorization	{{accessToken}}
Accept-Encoding	gzip, deflate, br
Connection	keep-alive

Screenshot 6

Headers tab — Authorization: {{accessToken}} confirmed

HTTPGet User Profile

SaveShare

GET{{BASE_URL}}/auth/meSend

DocsParamsAuthorizationHeaders10BodyScriptsSettings

Cookies

✓ Accept-Encoding	ⓘ gzip, deflate, br	
✓ Connection	ⓘ keep-alive	
✓ Authorization	{{accessToken}}	
Key	Value	Description

GET User Profile

GET{{BASE_URL}}/auth/me — Retrieve the authenticated user's profile

Request Details

Property	Value
Method	GET
URL	{{BASE_URL}}/auth/me
Auth	Bearer {{accessToken}}
Body	None
Expected	200 OK

Request URL

```
https://dummyjson.com/auth/me
```

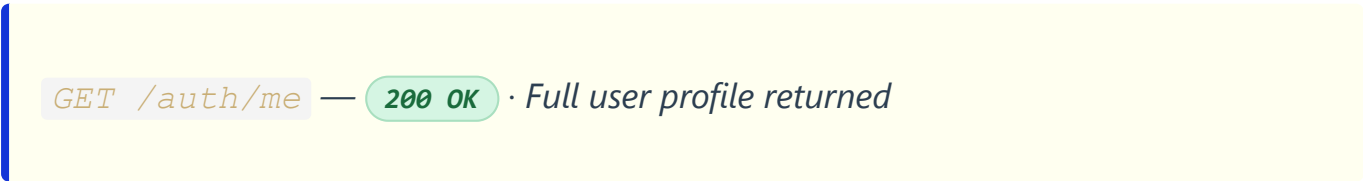
Procedure

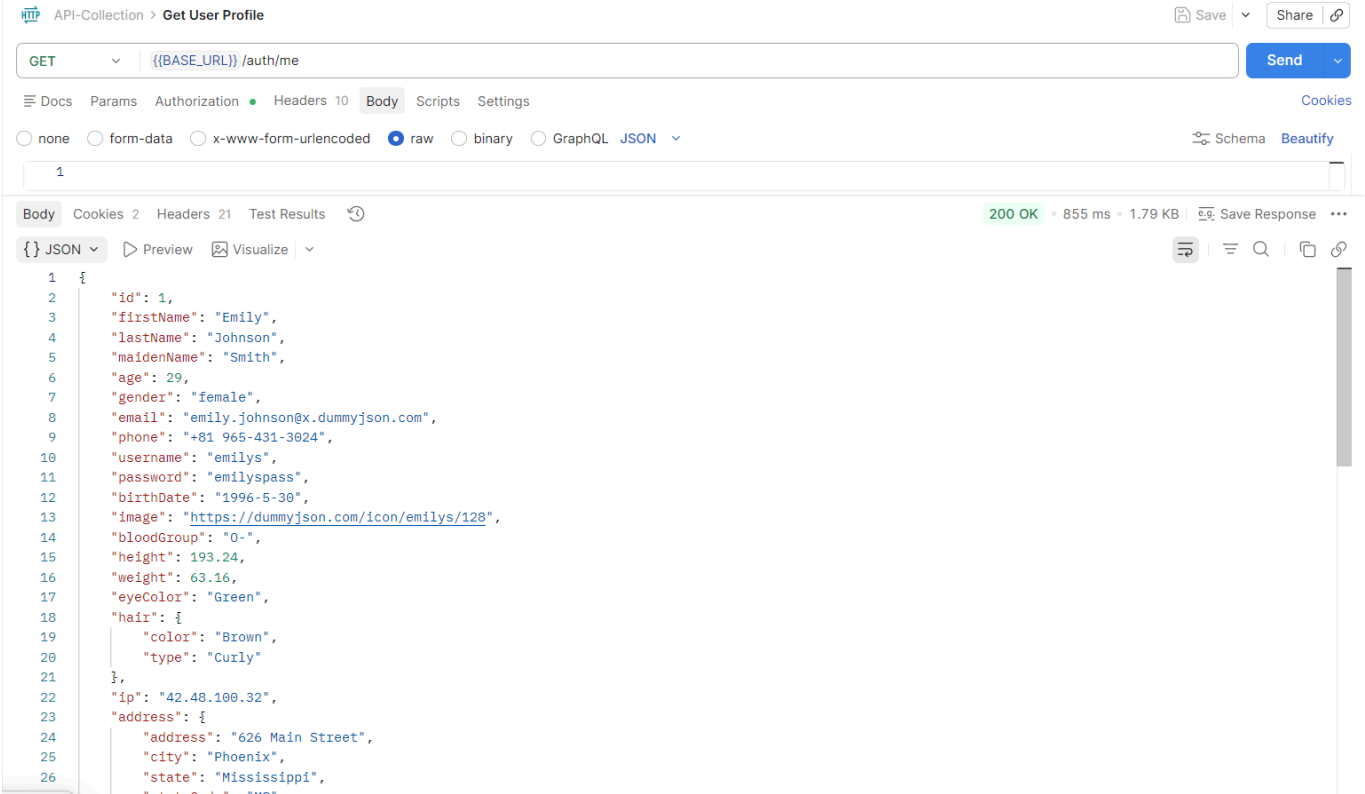
- 1. Open `Get User Profile`.
- 2. Click **Send**.
- 3. Confirm `200 OK` and the user profile JSON.

Key Response Fields

Field	Type	Example Value	Description
<code>id</code>	Int	<code>1</code>	Unique user identifier.
<code>firstName</code>	String	<code>Emily</code>	User's first name.
<code>lastName</code>	String	<code>Johnson</code>	User's last name.
<code>email</code>	String	<code>emily.johnson@x.dummyjson.com</code>	User's email address.
<code>username</code>	String	<code>emilys</code>	User's login username.
<code>role</code>	String	<code>admin</code>	User's role in the system.
<code>gender</code>	String	<code>female</code>	User's gender.
<code>age</code>	Int	<code>29</code>	User's age.

Screenshot 7





GET Products

GET

`{{BASE_URL}}/auth/products`

— Retrieve all products from the authenticated endpoint

Request Details

Property	Value
Method	GET
URL	<code>{{BASE_URL}}/auth/products</code>
Auth	Bearer <code>{{accessToken}}</code>
Body	None
Expected	200 OK

Request URL

```
https://dummyjson.com/auth/products
```

Procedure

- 1. In `OAuth-Login`, click **Add request**. Name it `Get Products`.
- 2. Set the method to **GET**. Enter the URL: `{{BASE_URL}}/auth/products`
- 3. Set Authorization to **Bearer Token** → `{{accessToken}}`.
- 4. Click **Send**. Confirm `200 OK` and a products array.

Key Response Fields (per product)

Field	Type	Example Value	Description
<code>id</code>	Int	<code>1</code>	Unique product identifier.
<code>title</code>	String	<code>Essence Mascara Lash Princess</code>	Product name.
<code>category</code>	String	<code>beauty</code>	Product category.
<code>price</code>	Float	<code>9.99</code>	Product price in USD.
<code>brand</code>	String	<code>Essence</code>	Product brand name.
<code>stock</code>	Int	<code>99</code>	Available stock quantity.
<code>availabilityStatus</code>	String	<code>In Stock</code>	Current availability status.

Screenshot 8

`GET /auth/products` — `200 OK` · Products array returned

The screenshot shows the Postman interface for a GET request to `{{BASE_URL}}/auth/products`. The response is a JSON array with one product object. The status is 200 OK, 612 ms, 7.85 KB.

```
{
  "products": [
    {
      "id": 1,
      "title": "Essence Mascara Lash Princess",
      "description": "The Essence Mascara Lash Princess is a popular mascara known for its volumizing and lengthening effects. Achieve dramatic lashes with this long-lasting and cruelty-free formula.",
      "category": "beauty",
      "price": 9.99,
      "discountPercentage": 10.48,
      "rating": 2.56,
      "stock": 99,
      "tags": [
        "beauty",
        "mascara"
      ],
      "brand": "Essence",
      "sku": "BEA-ESS-ESS-001",
      "weight": 4,
      "dimensions": {
        "width": 15.14,
        "height": 13.08,
        "depth": 22.99
      }
    }
  ]
}
```

Save Product ID via Script

9 Script — Save the first product's ID to the environment

Procedure

1. Open `Get Products`.
2. Click the **Scripts** tab → **Post-response**.
3. Paste the script below.
4. Click **Save**, then **Send**.

Post-Response Script

```
// Assert status code is 200
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});
```

```
// Save the first product's ID to the environment
let jsonData = pm.response.json();
pm.environment.set("productId", jsonData.products[0].id);
```

Screenshot 9

Scripts tab — `productId = products[0].id` saved to `Login_Environment`



Verify Environment Variables

10 **Verify** — Confirm all three variables are populated

Procedure

- 1. Click **Environments** in the left sidebar.
- 2. Click **Login_Environment**.
- 3. Confirm the values match the table below.

Expected Environment Variables

Variable	Expected Value	Set By
----------	----------------	--------

<code>BASE_URL</code>	<code>https://dummyjson.com</code>	Manual — Create Environment & Collection
-----------------------	------------------------------------	--

Variable	Expected Value	Set By
accessToken	eyJhbGci... (JWT)	Script — Extract Access Token via Script
productId	1	Script — Save Product ID via Script

Screenshot 10

`Login_Environment` — `BASE_URL`, `accessToken`, and `productId` all confirmed

Login_Environment

Fork0Share

Variable	Value	
BASE_URL	https://dummyjson.com	
accessToken	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXNlcm5hbWUiOiJlbWiseXMlLCJlbWFPbC6ImVtaWx5Lmpv...	
productId	1	
Add variable		

GET Product by ID

GET

`{{BASE_URL}}/auth/products/{{productId}}`

— Retrieve a single product by the saved productId

Request Details

Property	Value
Method	GET
URL	<code>{{BASE_URL}}/auth/products/{{productId}}</code>
Path Param	<code>{{productId}}</code>
Auth	Bearer <code>{{accessToken}}</code>

Property	Value
Expected	200 OK

Request URL

```
https://dummyjson.com/auth/products/1
```

Procedure

1. In OAuth-Login , click **Add request**. Name it `Get a Single Product` .

2. Set the method to **GET**. Enter the URL:
`{{BASE_URL}}/auth/products/{{productId}}`

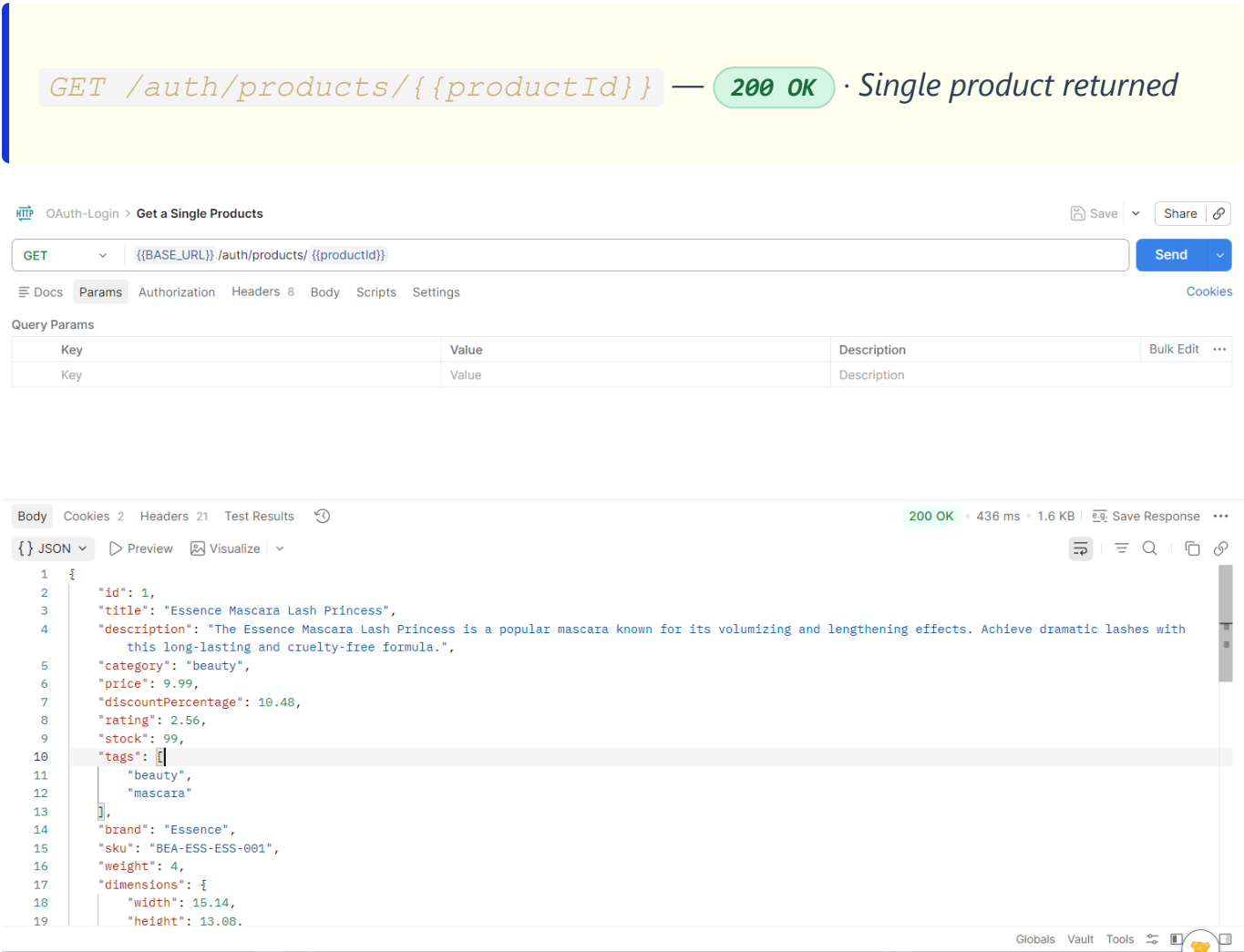
3. Set Authorization to **Bearer Token** → `{{accessToken}}` .

4. Click **Send**. Confirm `200 OK` and a single product object.

Key Response Fields

Field	Type	Example Value	Description
<code>id</code>	Int	<code>1</code>	Unique product identifier. Matches <code>productId</code> .
<code>title</code>	String	<code>Essence Mascara Lash Princess</code>	Product name.
<code>description</code>	String	—	Full product description.
<code>price</code>	Float	<code>9.99</code>	Product price in USD.
<code>discountPercentage</code>	Float	<code>10.48</code>	Discount applied to the product.
<code>rating</code>	Float	<code>2.56</code>	Average customer rating.
<code>stock</code>	Int	<code>99</code>	Available units in stock.
<code>sku</code>	String	<code>BEA-ESS-ESS-001</code>	Stock-keeping unit identifier.
<code>availabilityStatus</code>	String	<code>In Stock</code>	Current stock status.
<code>returnPolicy</code>	String	<code>No return policy</code>	Product return terms.

Screenshot 11



HTTP Status Code Reference

HTTP Status Code	Status Text	Applies To	Description
200	OK	All requests	The request succeeds. The response body contains the result.
400	Bad Request	POST	The request body is missing required fields or contains invalid data.

HTTP Status Code	Status Text	Applies To	Description
401	Unauthorized	All authenticated GET requests	The Bearer token is missing, expired, or invalid. Repeat from POST Login API .
404	Not Found	GET by ID	No resource exists for the given ID. Verify the <code>productId</code> variable.

dummyjson.com | Version 1.0 | OAuth Login · Bearer JWT Authentication